

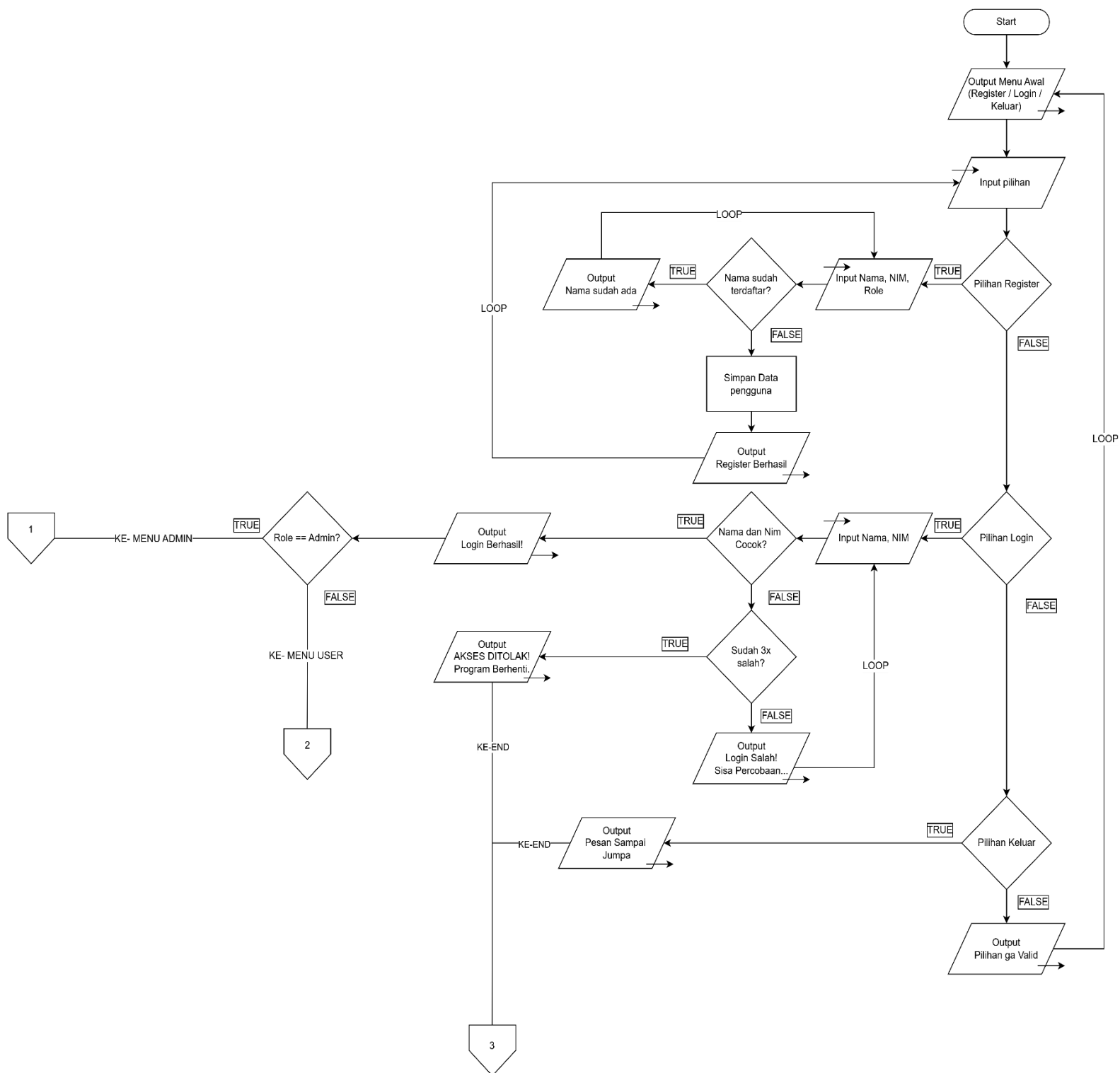
LAPORAN PRAKTIKUM
POSTTEST 7
ALGORITMA PEMROGRAMAN LANJUT



Disusun oleh:
Setriyani (2509106039)
Kelas (A2 '25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2025

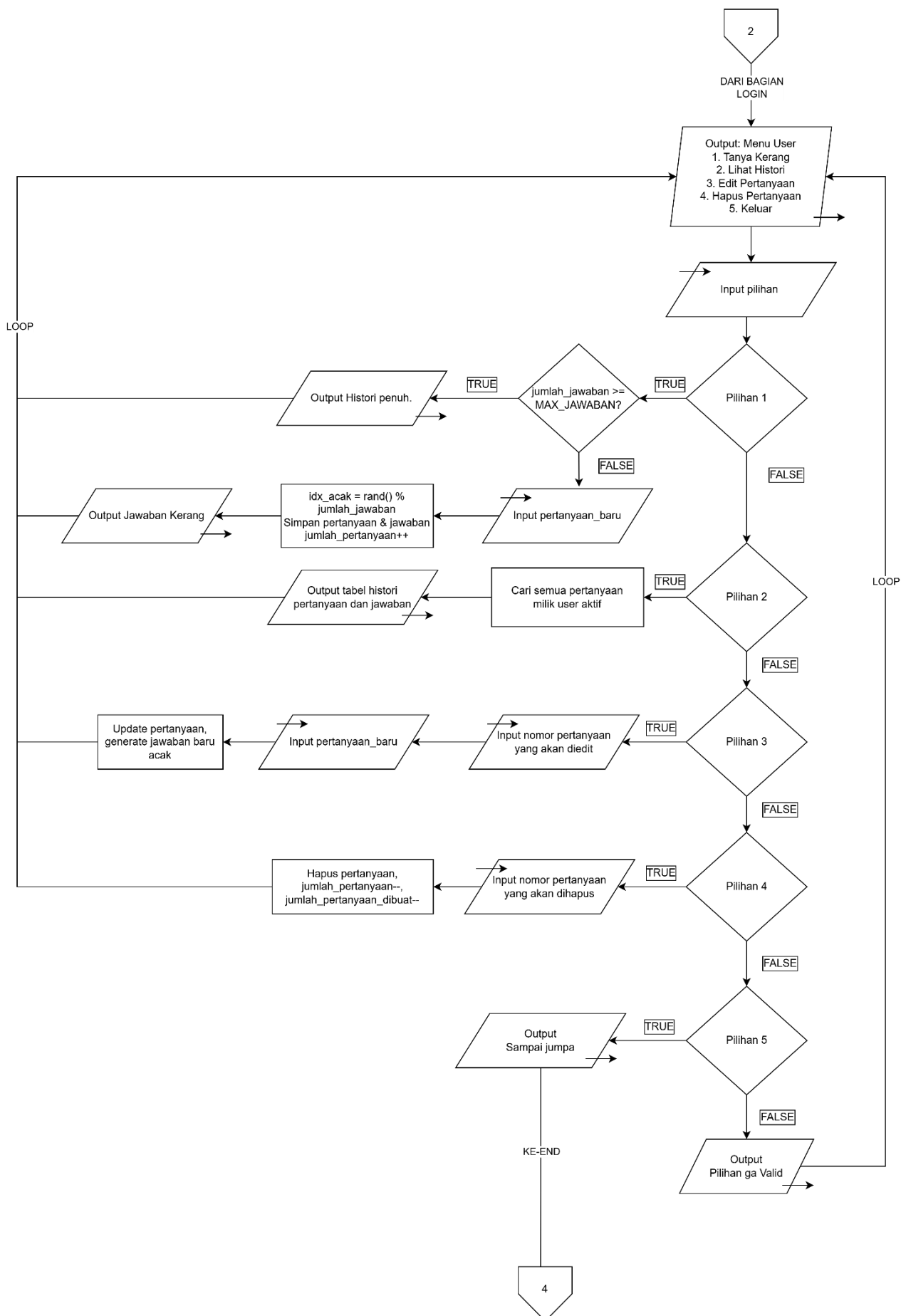
1. Flowchart



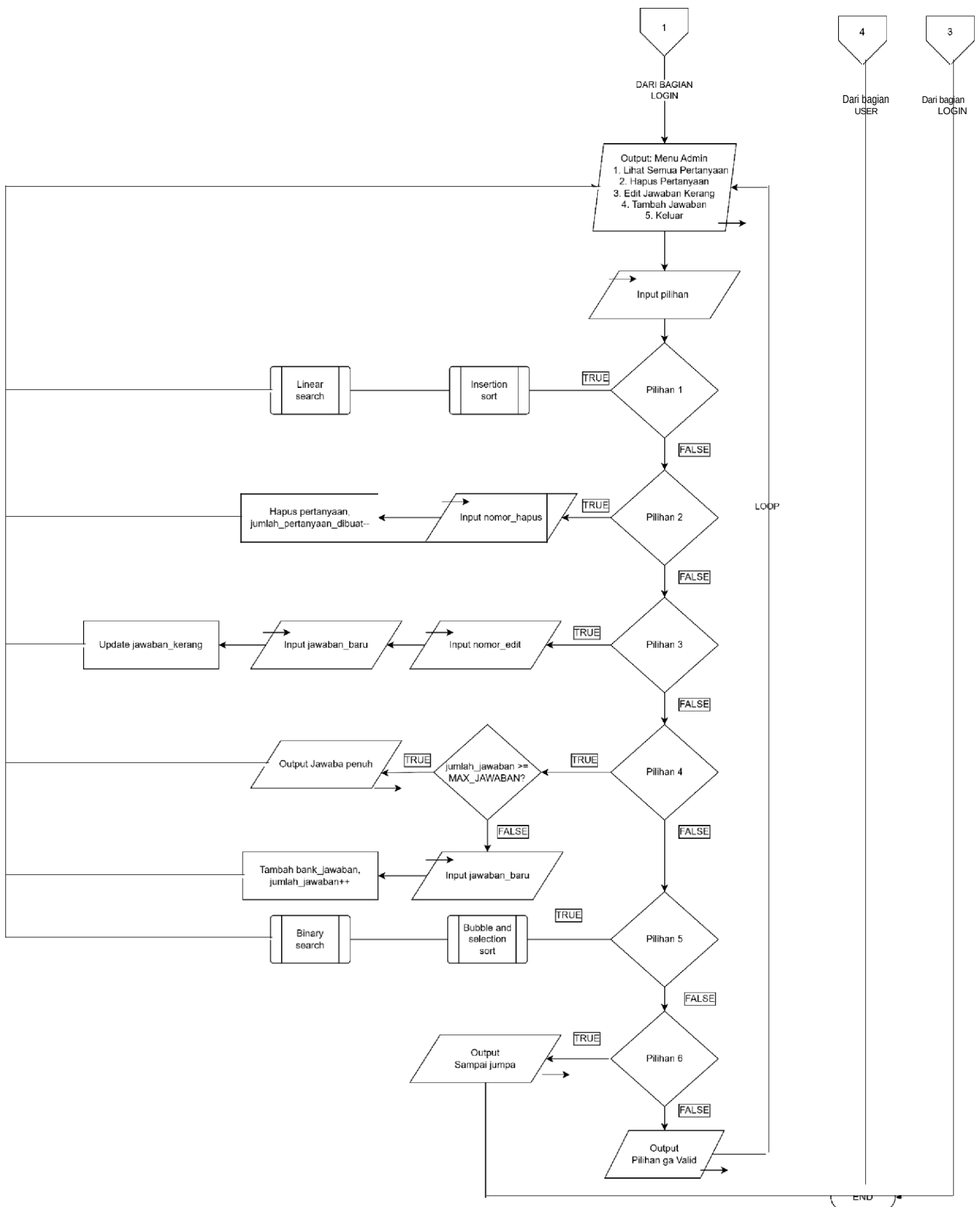
Gambar 1.1 Bagian login

Catatan sedikit:

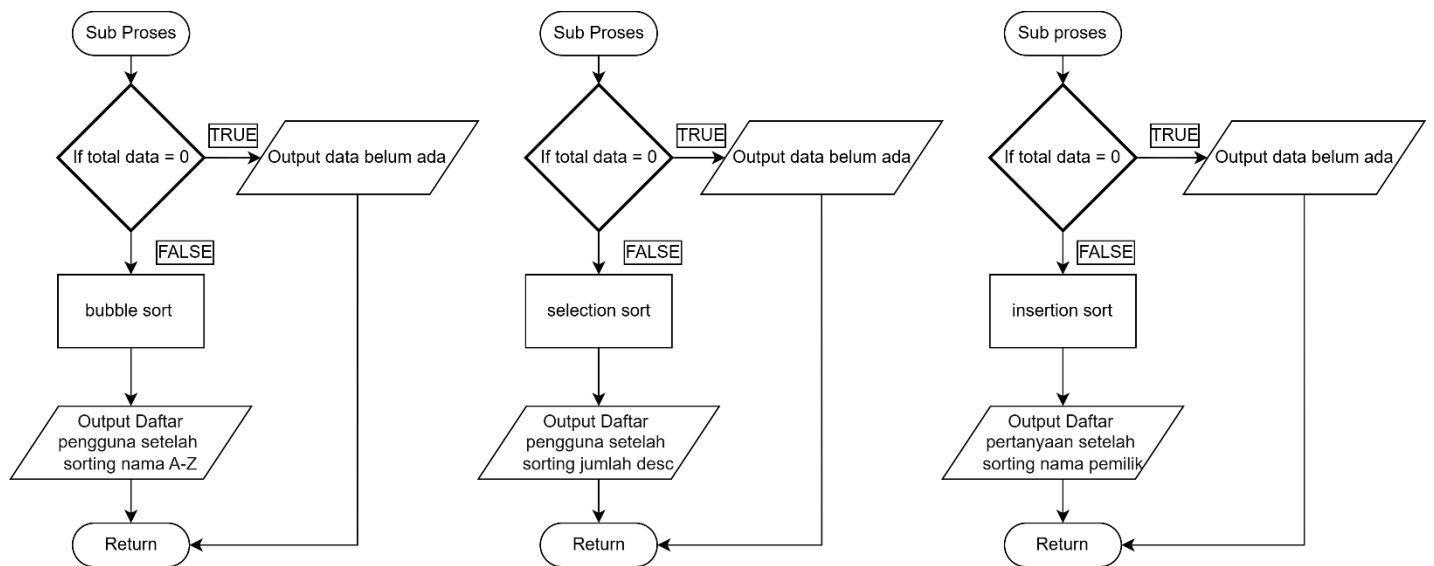
Abang/Mba itu bagian pilihan Login dan Pilihan keluar itu agak membingungkan, jadi saya tambahkan 2 kali yang tulisan “Ke-END” Biar alurnya bisa keliatan



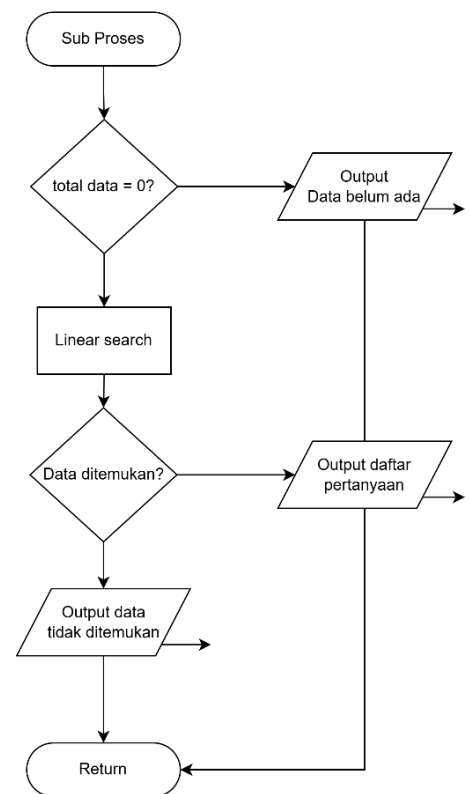
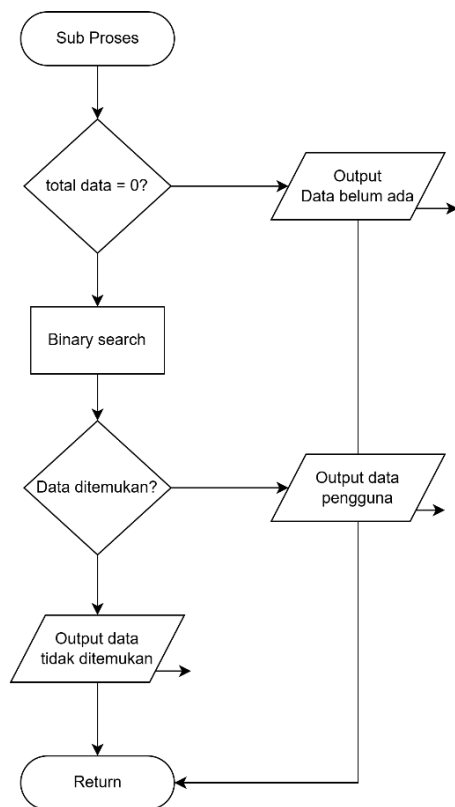
Gambar 1.2 Bagian Menu User



Gambar 1.3 Bagian Menu admin



Gambar 1.4 Sub proses sorting



Gambar 1.5 Sub Proses Searching

Penjelasan Flowchart:

Program dimulai dari START, lalu langsung menampilkan menu awal yang berisi pilihan Register, Login, dan Keluar. User diminta input pilihan.

Kalau pilih Register: user diminta input Nama, NIM, dan Role. Dicek dulu apakah role valid. Kalau tidak valid, tampil output "Role ga valid" lalu balik ke menu awal. Kalau valid, dicek apakah nama sudah terdaftar. Kalau sudah ada, tampil output "Nama sudah ada" lalu balik ke menu awal. Kalau belum ada, data profil disimpan lewat pointer ke struct ProfilPengguna, tampil output "Register Berhasil", lalu balik ke menu awal.

Kalau pilih Login: user diminta input Nama dan NIM. Dicek apakah nama dan NIM cocok. Kalau cocok, tampil output "Login Berhasil" lalu lanjut ke pengecekan role. Kalau salah, dicek sudah 3x percobaan belum. Kalau belum, tampil output "Login Salah! Sisa Percobaan..." lalu balik ke input nama dan NIM lagi. Kalau sudah 3x salah, tampil output "Akses Ditolak! Program Berhenti" dan program langsung selesai. Setelah login berhasil, dicek rolenya. Kalau role Admin, diarahkan ke Menu Admin. Kalau role User, diarahkan ke Menu User.

Di Menu Admin: user input pilihan 1 sampai 6. Pilihan 1 menampilkan tabel semua pertanyaan dari seluruh user yang sudah diurutkan berdasarkan nama pemilik menggunakan Insertion Sort, kemudian program menanyakan apakah user ingin mencari pertanyaan berdasarkan nama pemilik. Kalau ya, user diminta input nama yang dicari, lalu program menjalankan Linear Search untuk mencari semua pertanyaan milik nama tersebut dan menampilkan hasilnya. Kalau tidak ditemukan, tampil output "Pertanyaan tidak ditemukan". Pilihan 2 minta input nomor pertanyaan yang akan dihapus, lalu hapus pertanyaan dan update jumlah_pertanyaan_dibuat. Pilihan 3 minta input nomor edit lalu input jawaban baru, kemudian update jawaban_kerang. Pilihan 4 minta input jawaban baru lalu tambahkan ke bank_jawaban. Pilihan 5 menampilkan dua tabel daftar pengguna, tabel pertama menampilkan pengguna diurutkan nama A-Z menggunakan Bubble Sort, tabel kedua menampilkan pengguna diurutkan dari yang paling banyak pertanyaan menggunakan Selection Sort, kemudian program menanyakan apakah user ingin mencari pengguna berdasarkan nomor. Kalau ya, user diminta input nomor yang dicari, lalu program menjalankan Binary Search pada tabel nama A-Z dan menampilkan data pengguna yang ditemukan. Kalau tidak ditemukan, tampil output "Nomor tidak ditemukan". Pilihan 6 tampil output "Sampai jumpa" lalu kembali ke menu awal. Selain pilihan 1-6, tampil output "Pilihan ga valid". Semua pilihan yang selesai diproses akan kembali looping ke menu admin.

Di Menu User: user input pilihan 1 sampai 5. Pilihan 1 user input pertanyaan, lalu program generate jawaban acak, simpan pertanyaan dan jawaban, jumlah_pertanyaan_dibuat bertambah lewat pointer dereference, lalu tampil output jawaban kerang. Pilihan 2 program mencari semua pertanyaan milik user aktif secara rekursif, lalu tampilkan tabel histori pertanyaan dan jawaban. Pilihan 3 minta input nomor pertanyaan yang akan diedit, lalu input

pertanyaan baru, kemudian update pertanyaan dan jawaban lewat reference parameter. Pilihan 4 minta input nomor pertanyaan yang akan dihapus, lalu hapus pertanyaan dan kurangi jumlah_pertanyaan_dibuat lewat pointer dereference. Pilihan 5 tampil output "Sampai jumpa" lalu kembali ke menu awal. Selain pilihan 1-5, tampil output "Pilihan ga valid". Semua pilihan yang selesai diproses akan kembali looping ke menu user.

2. Deskripsi Singkat Program

Program ini nyiapin dulu tiga struct yaitu ProfilPengguna yang nyimpen nama dan NIM, struct Pengguna yang di dalamnya ada ProfilPengguna, role, sama jumlah_pertanyaan_dibuat, dan struct Pertanyaan yang nyimpen isi pertanyaan, jawaban kerang, sama nama pemiliknya. Program juga udah nyiapin 8 jawaban default di bank_jawaban lewat fungsi inisialisasiBankJawaban(). Setelah itu program nampilin layar selamat datang, lalu masuk ke looping menu awal yang isinya Register, Login, dan Keluar.

Kalau pilih Register, user diminta masukin Nama, NIM, dan Role. Kalau role bukan "user" atau "admin" muncul pesan role ga valid. Kalau valid, program ngecek apakah nama udah terdaftar lewat fungsi namaSudahAda(). Kalau udah ada muncul pesan nama udah ada. Kalau belum, program manggil fungsi isiProfilPengguna() yang nerima pointer ke struct ProfilPengguna milik pengguna baru, lalu ngisi nama dan NIM langsung ke alamat memori struct tersebut lewat arrow operator. Data pengguna baru disimpan ke vector daftar_pengguna dan muncul pesan register berhasil, lalu balik ke menu awal.

Kalau pilih Login, program manggil fungsi loginPengguna() yang minta user masukin Nama dan NIM, maksimal 3 kali percobaan. Setiap percobaan, program ngecek apakah nama dan NIM cocok dengan data yang tersimpan. Kalau cocok, login berhasil dan lanjut. Kalau salah dan belum 3x, muncul pesan sisa percobaan dan balik ke input lagi. Kalau udah salah 3x, muncul pesan akses ditolak dan program langsung berhenti. Setelah login berhasil, program ngecek role user. Kalau rolenya admin masuk ke menuAdmin(), kalau rolenya user masuk ke menuUser().

Di Menu Admin ada 6 pilihan yang diulang terus sampai pilih 6. Pilih 1 manggil lihatSemuaPertanyaan() yang nampilin tabel semua pertanyaan dari semua user lengkap dengan nomor, pemilik, pertanyaan, dan jawabannya; pada menu ini daftar pertanyaan diurutkan berdasarkan nama pemilik secara A-Z menggunakan Insertion Sort, setelah tabel ditampilkan program nanya apakah user mau cari pertanyaan by nama, kalau ya user diminta input nama yang dicari lalu program manggil linearSearchNamaPemilik() yang menelusuri satu per satu seluruh daftar pertanyaan untuk mencocokkan nama pemilik, kalau ditemukan ditampilkan dalam tabel, kalau tidak ditemukan tampil pesan pertanyaan tidak ditemukan. Pilih 2 manggil hapusPertanyaanAdmin() yang nampilin tabel pertanyaan lalu minta input nomor yang mau dihapus, setelah dihapus program manggil kurangiJumlahPertanyaan() yang nerima pointer ke jumlah_pertanyaan_dibuat pemiliknya dan langsung ngurangi nilainya lewat dereference operator. Pilih 3 manggil editJawabanAdmin() yang nampilin tabel pertanyaan beserta jawaban sekarang, lalu minta input nomor dan jawaban baru, perubahannya dikirim lewat fungsi perbaruiJawaban() yang nerima reference parameter

sehingga jawaban aslinya langsung berubah. Pilih 4 manggil tambahJawabanKerang() yang nampilin daftar jawaban yang ada lalu minta input jawaban baru, kalau diisi 0 berarti batal, kalau kosong ditolak, kalau diisi normal maka jawaban baru ditambahkan ke bank_jawaban. Pilih 5 manggil lihatDaftarPengguna() yang menampilkan dua jenis tabel: pertama, daftar pengguna yang diurutkan nama A-Z menggunakan Bubble Sort, dan kedua, daftar pengguna paling aktif yang diurutkan berdasarkan jumlah pertanyaan terbanyak menggunakan Selection Sort, setelah kedua tabel ditampilkan program nanya apakah user mau cari pengguna by nomor, kalau ya user diminta input nomor yang dicari lalu program manggil binarySearchNomorPengguna() yang nerima pointer ke vector daftar pengguna dan mencari dengan membagi rentang pencarian jadi dua terus sampai ketemu atau habis, kalau ditemukan tampil data nama, role, dan jumlah pertanyaan pengguna tersebut, kalau tidak ditemukan tampil pesan nomor tidak ditemukan. Pilih 6 muncul pesan bye dan keluar dari menu admin, balik ke menu awal.

Di Menu User juga ada 5 pilihan yang diulang terus sampai pilih 5. Pilih 1 manggil tanyaKerang() yang minta user masukin pertanyaan, lalu program generate jawaban acak lewat fungsi ambilJawabanAcak(), simpan pertanyaan dan jawaban ke vector, lalu manggil tambahJumlahPertanyaan() yang nerima pointer ke jumlah_pertanyaan_dibuat dan langsung nambah nilainya lewat dereference operator, lalu nampilin jawaban kerang. Pilih 2 manggil lihatHistori() yang nampilin tabel histori pertanyaan milik user aktif menggunakan fungsi rekursif tampilBarisRekursif() yang menelusuri daftar pertanyaan satu per satu sampai semua index habis. Pilih 3 manggil editPertanyaan() yang nampilin tabel pertanyaan milik user, minta input nomor yang mau diedit lalu input pertanyaan baru, perubahannya dikirim lewat perbaruiPertanyaan() dan perbaruiJawaban() yang keduanya nerima reference parameter sehingga data aslinya langsung terupdate. Pilih 4 manggil hapusPertanyaanUser() yang nampilin tabel pertanyaan milik user, minta input nomor yang mau dihapus, lalu hapus pertanyaannya dan manggil kurangiJumlahPertanyaan() lewat pointer untuk ngurangi counter. Pilih 5 muncul pesan bye dan keluar dari menu user, balik ke menu awal.

Kalau di menu awal pilih Keluar, muncul pesan sampai jumpa dan program selesai.

3. Source Code

BAGIAN AWAL/LOGIN/REGISTER

```
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <cstdlib>
#include <ctime>
#include <stdexcept>
#include "kerang.h"
using namespace std;

struct ProfilPengguna {
    string nama;
    string nim;
};
```

```

struct Pengguna {
    ProfilPengguna profil;
    string role;
    int jumlah_pertanyaan_dibuat;
};

struct Pertanyaan {
    string isi_pertanyaan;
    string jawaban_kerang;
    string nama_pemilik;
};

vector<Pengguna>    daftar_pengguna;
vector<Pertanyaan> daftar_pertanyaan;
vector<string>      bank_jawaban;

void inisialisasiBankJawaban() {
    bank_jawaban.push_back("Mungkin suatu hari nanti...");
    bank_jawaban.push_back("Tidak.");
    bank_jawaban.push_back("Coba lagi nanti.");
    bank_jawaban.push_back("Tentu saja.");
    bank_jawaban.push_back("Kerang tidak menjawab pertanyaan itu.");
    bank_jawaban.push_back("Sepertinya iya.");
    bank_jawaban.push_back("Sepertinya tidak.");
    bank_jawaban.push_back("Sudah jelas jawabannya.");
}

void tampilHeaderTabel(int lebar_col1, string judul_col1) {
    cout << "+-" << setfill('-') << setw(4) << "-"
         << "+-" << setw(lebar_col1) << "-" << "+" << endl;
    cout << setfill(' ');
    cout << "| " << left << setw(4) << "No"
         << "| " << setw(lebar_col1) << judul_col1 << "|" << endl;
    cout << "+-" << setfill('-') << setw(4) << "-"
         << "+-" << setw(lebar_col1) << "-" << "+" << endl;
    cout << setfill(' ');
}

void tampilHeaderTabel(int lebar_col1, int lebar_col2,
                      string judul_col1, string judul_col2) {
    cout << "+-" << setfill('-') << setw(4) << "-"
         << "+-" << setw(lebar_col1) << "-"
         << "+-" << setw(lebar_col2) << "-" << "+" << endl;
    cout << setfill(' ');
    cout << "| " << left << setw(4) << "No"
         << "| " << setw(lebar_col1) << judul_col1
         << "| " << setw(lebar_col2) << judul_col2 << "|" << endl;
    cout << "+-" << setfill('-') << setw(4) << "-"
         << "+-" << setw(lebar_col1) << "-"
         << "+-" << setw(lebar_col2) << "-" << "+" << endl;
    cout << setfill(' ');
}

void tampilHeaderTabel(int lebar_col1, int lebar_col2, int lebar_col3,
                      string judul_col1, string judul_col2, string judul_col3) {
    cout << "+-" << setfill('-') << setw(4) << "-"
         << "+-" << setw(lebar_col1) << "-"

```

```

        << "+-" << setw(lebar_col2) << "-"
        << "+-" << setw(lebar_col3) << "-" << "+" << endl;
    cout << setfill(' ');
    cout << "| " << left << setw(4) << "No"
        << "| " << setw(lebar_col1) << judul_col1
        << "| " << setw(lebar_col2) << judul_col2
        << "| " << setw(lebar_col3) << judul_col3 << "|" << endl;
    cout << "+-" << setfill('-') << setw(4) << "-"
        << "+-" << setw(lebar_col1) << "-"
        << "+-" << setw(lebar_col2) << "-"
        << "+-" << setw(lebar_col3) << "-" << "+" << endl;
    cout << setfill(' ');
}

void tampilGarisBawahTabel(int lebar_col1, int lebar_col2) {
    cout << "+-" << setfill('-') << setw(4) << "-"
        << "+-" << setw(lebar_col1) << "-"
        << "+-" << setw(lebar_col2) << "-" << "+" << endl;
    cout << setfill(' ');
}

void tampilGarisBawahTabel(int lebar_col1, int lebar_col2, int lebar_col3) {
    cout << "+-" << setfill('-') << setw(4) << "-"
        << "+-" << setw(lebar_col1) << "-"
        << "+-" << setw(lebar_col2) << "-"
        << "+-" << setw(lebar_col3) << "-" << "+" << endl;
    cout << setfill(' ');
}

void tampilBarisRekursif(int index_saati_ini, string nama_pemilik,
                        int &nomor_urut, int lebar_col1, int lebar_col2) {
    if (index_saati_ini >= (int)daftar_pertanyaan.size()) return;
    if (daftar_pertanyaan[index_saati_ini].nama_pemilik == nama_pemilik) {
        nomor_urut++;
        string pertanyaan = daftar_pertanyaan[index_saati_ini].isi_pertanyaan;
        string jawaban = daftar_pertanyaan[index_saati_ini].jawaban_kerang;
        if ((int)pertanyaan.length() > lebar_col1 - 2)
            pertanyaan = pertanyaan.substr(0, lebar_col1 - 5) + "...";
        if ((int)jawaban.length() > lebar_col2 - 2)
            jawaban = jawaban.substr(0, lebar_col2 - 5) + "...";
        cout << "| " << left << setw(4) << nomor_urut
            << "| " << setw(lebar_col1) << pertanyaan
            << "| " << setw(lebar_col2) << jawaban << "|" << endl;
    }
    tampilBarisRekursif(index_saati_ini + 1, nama_pemilik, nomor_urut, lebar_col1,
lebar_col2);
}

string ambilJawabanAcak(vector<string> &daftar_jawaban) {
    int index_acak = rand() % (int)daftar_jawaban.size();
    return daftar_jawaban[index_acak];
}

bool namaSudahAda(string nama_dicari) {
    for (int i = 0; i < (int)daftar_pengguna.size(); i++) {
        if (daftar_pengguna[i].profil.nama == nama_dicari) return true;
    }
    return false;
}

```

```

}

void isiProfilPengguna(ProfilPengguna *ptr_profil, string nama_masukan, string
nim_masukan) {
    ptr_profil->nama = nama_masukan;
    ptr_profil->nim = nim_masukan;
}

void tambahJumlahPertanyaan(int *ptr_jumlah) {
    (*ptr_jumlah)++;
}

void kurangiJumlahPertanyaan(int *ptr_jumlah) {
    if (*ptr_jumlah > 0) (*ptr_jumlah)--;
}

void perbaruiJawaban(string &jawaban_lama, string jawaban_baru) {
    jawaban_lama = jawaban_baru;
}

void perbaruiPertanyaan(string &isi_lama, string isi_baru) {
    isi_lama = isi_baru;
}

void bubbleSortNamaAscending(vector<Pengguna> &daftar) {
    int ukuran = (int)daftar.size();
    for (int i = 0; i < ukuran - 1; i++) {
        for (int j = 0; j < ukuran - i - 1; j++) {
            string nama_kiri = daftar[j].profil.nama;
            string nama_kanan = daftar[j + 1].profil.nama;
            for (int k = 0; k < (int)nama_kiri.length(); k++) nama_kiri[k] =
tolower(nama_kiri[k]);
            for (int k = 0; k < (int)nama_kanan.length(); k++) nama_kanan[k] =
tolower(nama_kanan[k]);
            if (nama_kiri > nama_kanan) swap(daftar[j], daftar[j + 1]);
        }
    }
}

void selectionSortJumlahDescending(vector<Pengguna> &daftar) {
    int ukuran = (int)daftar.size();
    for (int i = 0; i < ukuran - 1; i++) {
        int index_terbesar = i;
        for (int j = i + 1; j < ukuran; j++) {
            if (daftar[j].jumlah_pertanyaan_dibuat >
daftar[index_terbesar].jumlah_pertanyaan_dibuat)
                index_terbesar = j;
        }
        if (index_terbesar != i) swap(daftar[i], daftar[index_terbesar]);
    }
}

void insertionSortNamaPemilikAscending(vector<Pertanyaan> &daftar) {
    int ukuran = (int)daftar.size();
    for (int i = 1; i < ukuran; i++) {
        Pertanyaan kunci = daftar[i];
        int j = i - 1;
        while (j >= 0 && daftar[j].nama_pemilik < kunci.nama_pemilik) {

```

```

        daftar[j + 1] = daftar[j];
        j--;
    }
    daftar[j + 1] = kunci;
}
}

int binarySearchNomorPengguna(vector<Pengguna> *ptr_daftar, int nomor_dicari) {
    int batas_kiri = 0;
    int batas_kanan = (int)ptr_daftar->size() - 1;
    while (batas_kiri <= batas_kanan) {
        int tengah = (batas_kiri + batas_kanan) / 2;
        if (tengah + 1 == nomor_dicari) return tengah;
        else if (tengah + 1 < nomor_dicari) batas_kiri = tengah + 1;
        else batas_kanan = tengah - 1;
    }
    return -1;
}

void linearSearchNamaPemilik(vector<Pertanyaan> *ptr_daftar, string nama_dicari) {
    string nama_dicari_lower = nama_dicari;
    for (int i = 0; i < (int)nama_dicari_lower.length(); i++)
        nama_dicari_lower[i] = tolower(nama_dicari_lower[i]);

    bool ditemukan = false;
    tampilHeaderTabel(15, 28, 28, "Pemilik", "Pertanyaan", "Jawaban");
    for (int i = 0; i < (int)ptr_daftar->size(); i++) {
        string nama_lower = (*ptr_daftar)[i].nama_pemilik;
        for (int j = 0; j < (int)nama_lower.length(); j++) nama_lower[j] =
tolower(nama_lower[j]);
        if (nama_lower == nama_dicari_lower) {
            ditemukan = true;
            string pemilik = (*ptr_daftar)[i].nama_pemilik;
            string pertanyaan = (*ptr_daftar)[i].isi_pertanyaan;
            string jawaban = (*ptr_daftar)[i].jawaban_kerang;
            if ((int)pemilik.length() > 13) pemilik = pemilik.substr(0, 10) +
"...";
            if ((int)pertanyaan.length() > 26) pertanyaan = pertanyaan.substr(0, 23) +
"...";
            if ((int)jawaban.length() > 26) jawaban = jawaban.substr(0, 23) +
"...";
            cout << "| " << left << setw(4) << i + 1
                << "| " << setw(15) << pemilik
                << "| " << setw(28) << pertanyaan
                << "| " << setw(28) << jawaban << "|" << endl;
        }
    }
    tampilGarisBawahTabel(15, 28, 28);
    if (!ditemukan) cout << " Pertanyaan milik \"" << nama_dicari << "\" tidak
ditemukan." << endl;
    cout << endl;
}

void registerPengguna() {
    string nama_input, nim_input, role_input;
    cout << "===== " << endl;
    cout << " REGISTER " << endl;
    cout << "===== " << endl;
}

```

```

cout << "    Nama            : "; getline(cin, nama_input);
cout << "    Password (NIM)   : "; getline(cin, nim_input);
cout << "    Role (user/admin): "; getline(cin, role_input);

for (int i = 0; i < (int)role_input.length(); i++) role_input[i] =
tolower(role_input[i]);

try {
    validasiInputTidakKosong(nama_input, "Nama");
    validasiInputTidakKosong(nim_input, "NIM");
    validasiRole(role_input);

    if (namaSudahAda(nama_input)) {
        throw runtime_error("Nama udah ada, ganti yang lain.");
    }

    Pengguna pengguna_baru;
    isiProfilPengguna(&pengguna_baru.profil, nama_input, nim_input);
    pengguna_baru.role = role_input;
    pengguna_baru.jumlah_pertanyaan_dibuat = 0;
    daftar_pengguna.push_back(pengguna_baru);
    cout << "    You berhasil register sebagai " << role_input << ". Silakan login."
<< endl << endl;

} catch (const invalid_argument &e) {
    cout << "    [ERROR] " << e.what() << endl << endl;
} catch (const runtime_error &e) {
    cout << "    [ERROR] " << e.what() << endl << endl;
}
}

int loginPengguna(vector<Pengguna> &daftar_pengguna) {
    int percobaan = 0;
    bool login_berhasil = false;
    int index_pengguna = -1;

    cout << "===== " << endl;
    cout << "                LOGIN                " << endl;
    cout << "===== " << endl;

    while (percobaan < 3) {
        percobaan++;
        string nama_input, nim_input;
        cout << "    Percobaan ke-" << percobaan << " dari 3" << endl;
        cout << "    Nama            : "; getline(cin, nama_input);
        cout << "    Password (NIM): "; getline(cin, nim_input);

        try {
            validasiInputTidakKosong(nama_input, "Nama");
            validasiInputTidakKosong(nim_input, "NIM");
        } catch (const invalid_argument &e) {
            cout << "    [ERROR] " << e.what() << endl << endl;
            continue;
        }

        string nama_input_lower = nama_input;
        for (int i = 0; i < (int)nama_input_lower.length(); i++)
            nama_input_lower[i] = tolower(nama_input_lower[i]);
    }
}

```

```
        for (int i = 0; i < (int)daftar_pengguna.size(); i++) {
            string nama_tersimpan_lower = daftar_pengguna[i].profil.nama;
            for (int j = 0; j < (int)nama_tersimpan_lower.length(); j++)
                nama_tersimpan_lower[j] = tolower(nama_tersimpan_lower[j]);
            if (nama_tersimpan_lower == nama_input_lower &&
daftar_pengguna[i].profil.nim == nim_input) {
                login_berhasil = true;
                index_pengguna = i;
            }
        }
        if (login_berhasil) break;
        if (percobaan < 3)
            cout << "    Nama atau NIM salah. Sisa: " << (3 - percobaan) << "x" << endl <<
endl;
    }
    return index_pengguna;
}
```

BAGIAN MENU USER/ADMIN

```
void tanyaKerang(int index_pengguna) {
    string pertanyaan_baru;
    cout << "  Tanyakan apapun pada kerang ajaib..." << endl;
    cout << "  Pertanyaanmu: "; getline(cin, pertanyaan_baru);

    try {
        validasiInputTidakKosong(pertanyaan_baru, "Pertanyaan");

        string jawaban_terpilih = ambilJawabanAcak(bank_jawaban);

        Pertanyaan pertanyaan_baru_obj;
        pertanyaan_baru_obj.isi_pertanyaan = pertanyaan_baru;
        pertanyaan_baru_obj.jawaban_kerang = jawaban_terpilih;
        pertanyaan_baru_obj.nama_pemilik =
daftar_pengguna[index_pengguna].profil.nama;
        daftar_pertanyaan.push_back(pertanyaan_baru_obj);
        tambahJumlahPertanyaan(&daftar_pengguna[index_pengguna].jumlah_pertanyaan_dibuat
);

        cout << endl;
        cout << "  Kerang Ajaib berkata.." << endl;
        cout << "  >> \"" << jawaban_terpilih << "\"" << endl << endl;

    } catch (const invalid_argument &e) {
        cout << "  [ERROR] " << e.what() << endl << endl;
    }
}

void lihatHistori(int index_pengguna) {
    string nama_aktif = daftar_pengguna[index_pengguna].profil.nama;
    int jumlah_milik = daftar_pengguna[index_pengguna].jumlah_pertanyaan_dibuat;

    cout << "  Histori punya " << nama_aktif << " (" << jumlah_milik << " pertanyaan)"
<< endl;

    try {
        validasiVectorTidakKosong(jumlah_milik, "Histori pertanyaan");
        tampilHeaderTabel(30, 30, "Pertanyaan", "Jawaban Kerang");
        int nomor_urut = 0;
        tampilBarisRekursif(0, nama_aktif, nomor_urut, 30, 30);
        tampilGarisBawahTabel(30, 30);
        cout << endl;
    } catch (const runtime_error &e) {
        cout << "  [ERROR] " << e.what() << " Tanya kerang dulu kocak." << endl << endl;
    }
}

vector<int> kumpulkanIndexMilikUser(string nama_pemilik) {
    vector<int> daftar_index;
    for (int i = 0; i < (int)daftar_pertanyaan.size(); i++) {
        if (daftar_pertanyaan[i].nama_pemilik == nama_pemilik)
            daftar_index.push_back(i);
    }
}
```



```

    return daftar_index;
}

void tampilTabelPertanyaanUser(vector<int> daftar_index) {
    tampilHeaderTabel(45, "Pertanyaan");
    for (int i = 0; i < (int)daftar_index.size(); i++) {
        string pertanyaan = daftar_pertanyaan[daftar_index[i]].isi_pertanyaan;
        if ((int)pertanyaan.length() > 43) pertanyaan = pertanyaan.substr(0, 40) +
"...";
        cout << "| " << left << setw(4) << i + 1
            << "| " << setw(45) << pertanyaan << "| " << endl;
    }
    cout << "+-" << setfill('-') << setw(4) << "- "
        << "+-" << setw(45) << "- " << "+" << endl;
    cout << setfill(' ');
}

void editPertanyaan(int index_pengguna) {
    string nama_aktif = daftar_pengguna[index_pengguna].profil.nama;
    vector<int> daftar_index = kumpulkanIndexMilikUser(nama_aktif);

    try {
        validasiVectorTidakKosong((int)daftar_index.size(), "Pertanyaan");
        tampilTabelPertanyaanUser(daftar_index);

        int nomor_pilihan;
        cout << "    Pilih nomor: "; cin >> nomor_pilihan; cin.ignore();
        validasiNomorPilihan(nomor_pilihan, 1, (int)daftar_index.size());

        string pertanyaan_baru;
        cout << "    Pertanyaan baru: "; getline(cin, pertanyaan_baru);
        validasiInputTidakKosong(pertanyaan_baru, "Pertanyaan baru");

        string jawaban_baru = ambilJawabanAcak(bank_jawaban);
        int index_asli = daftar_index[nomor_pilihan - 1];
        perbaruiPertanyaan(daftar_pertanyaan[index_asli].isi_pertanyaan,
pertanyaan_baru);
        perbaruiJawaban(daftar_pertanyaan[index_asli].jawaban_kerang, jawaban_baru);

        cout << "    Pertanyaan diperbarui!" << endl;
        cout << "    >> Kerang: \"" << jawaban_baru << "\"" << endl << endl;

    } catch (const runtime_error &e) {
        cout << "    [ERROR] " << e.what() << endl << endl;
    } catch (const out_of_range &e) {
        cout << "    [ERROR] " << e.what() << endl << endl;
    } catch (const invalid_argument &e) {
        cout << "    [ERROR] " << e.what() << endl << endl;
    }
}

void hapusPertanyaanUser(int index_pengguna) {
    string nama_aktif = daftar_pengguna[index_pengguna].profil.nama;
    vector<int> daftar_index = kumpulkanIndexMilikUser(nama_aktif);

    try {
        validasiVectorTidakKosong((int)daftar_index.size(), "Pertanyaan");
        tampilTabelPertanyaanUser(daftar_index);
    }
}

```

```

    int nomor_pilihan;
    cout << " Pilih nomor: "; cin >> nomor_pilihan; cin.ignore();
    validasiNomorPilihan(nomor_pilihan, 1, (int)daftar_index.size());

    int index_asli = daftar_index[nomor_pilihan - 1];
    daftar_pertanyaan.erase(daftar_pertanyaan.begin() + index_asli);
    kurangiJumlahPertanyaan(&daftar_pengguna[index_pengguna].jumlah_pertanyaan_dibuat);
    cout << " Pertanyaan dihapus." << endl << endl;

} catch (const runtime_error &e) {
    cout << " [ERROR] " << e.what() << endl << endl;
} catch (const out_of_range &e) {
    cout << " [ERROR] " << e.what() << endl << endl;
}
}

void lihatSemuaPertanyaan() {
    try {
        validasiVectorTidakKosong((int)daftar_pertanyaan.size(), "Data pertanyaan");

        cout << " === Nama pemilik A-Z (Insertion Sort) ===" << endl;
        vector<Pertanyaan> daftar_terurut = daftar_pertanyaan;
        insertionSortNamaPemilikAscending(daftar_terurut);

        tampilHeaderTabel(15, 28, 28, "Pemilik", "Pertanyaan", "Jawaban");
        for (int i = 0; i < (int)daftar_terurut.size(); i++) {
            string pemilik = daftar_terurut[i].nama_pemilik;
            string pertanyaan = daftar_terurut[i].isi_pertanyaan;
            string jawaban = daftar_terurut[i].jawaban_kerang;
            if ((int)pekilik.length() > 13) pekilik = pekilik.substr(0, 10) +
            "...";
            if ((int)pertanyaan.length() > 26) pertanyaan = pertanyaan.substr(0, 23) +
            "...";
            if ((int)jawaban.length() > 26) jawaban = jawaban.substr(0, 23) +
            "...";

            cout << "| " << left << setw(4) << i + 1
                << "| " << setw(15) << pekilik
                << "| " << setw(28) << pertanyaan
                << "| " << setw(28) << jawaban << "|" << endl;
        }
        tampilGarisBawahTabel(15, 28, 28);
        cout << endl;

        string pilihan_cari;
        cout << " Mau cari pertanyaan by nama? (y/n): "; getline(cin, pilihan_cari);
        if (pilihan_cari == "y" || pilihan_cari == "Y") {
            string nama_dicari;
            cout << " Cari nama: "; getline(cin, nama_dicari);
            linearSearchNamaPemilik(&daftar_terurut, nama_dicari);
        }

    } catch (const runtime_error &e) {
        cout << " [ERROR] " << e.what() << endl << endl;
    }
}

```

```

void lihatDaftarPengguna() {
    try {
        validasiVectorTidakKosong((int)daftar_pengguna.size(), "Data pengguna");

        vector<Pengguna> terurut_nama = daftar_pengguna;
        vector<Pengguna> terurut_aktif = daftar_pengguna;
        bubbleSortNamaAscending(terurut_nama);
        selectionSortJumlahDescending(terurut_aktif);

        cout << " === Nama A-Z (Bubble Sort) ===" << endl;
        tampilHeaderTabel(20, 10, 12, "Nama", "Role", "Jml Pertanyaan");
        for (int i = 0; i < (int)terurut_nama.size(); i++) {
            string nama = terurut_nama[i].profil.nama;
            if ((int)nama.length() > 18) nama = nama.substr(0, 15) + "...";
            cout << "| " << left << setw(4) << i + 1
                << "| " << setw(20) << nama
                << "| " << setw(10) << terurut_nama[i].role
                << "| " << setw(12) << terurut_nama[i].jumlah_pertanyaan_dibuat << "| "
<< endl;
        }
        tampilGarisBawahTabel(20, 10, 12);
        cout << endl;

        cout << " === Paling Aktif (Selection Sort) ===" << endl;
        tampilHeaderTabel(20, 10, 12, "Nama", "Role", "Jml Pertanyaan");
        for (int i = 0; i < (int)terurut_aktif.size(); i++) {
            string nama = terurut_aktif[i].profil.nama;
            if ((int)nama.length() > 18) nama = nama.substr(0, 15) + "...";
            cout << "| " << left << setw(4) << i + 1
                << "| " << setw(20) << nama
                << "| " << setw(10) << terurut_aktif[i].role
                << "| " << setw(12) << terurut_aktif[i].jumlah_pertanyaan_dibuat << "| "
<< endl;
        }
        tampilGarisBawahTabel(20, 10, 12);
        cout << endl;

        string pilihan_cari;
        cout << " Mau cari pengguna by nomor? (y/n): "; getline(cin, pilihan_cari);
        if (pilihan_cari == "y" || pilihan_cari == "Y") {
            int nomor_dicari;
            cout << " Cari nomor: "; cin >> nomor_dicari; cin.ignore();
            int hasil = binarySearchNomorPengguna(&terurut_nama, nomor_dicari);
            if (hasil == -1) {
                cout << " Nomor " << nomor_dicari << " tidak ditemukan." << endl;
            } else {
                cout << " Ditemukan!" << endl;
                cout << " Nama : " << terurut_nama[hasil].profil.nama << endl;
                cout << " Role : " << terurut_nama[hasil].role << endl;
                cout << " Jumlah Pertanyaan: " <<
terurut_nama[hasil].jumlah_pertanyaan_dibuat << endl;
            }
            cout << endl;
        }

    } catch (const runtime_error &e) {
        cout << " [ERROR] " << e.what() << endl << endl;
    }
}

```

```

}

void hapusPertanyaanAdmin() {
    try {
        validasiVectorTidakKosong((int)daftar_pertanyaan.size(), "Data pertanyaan");

        tampilHeaderTabel(15, 38, "Pemilik", "Pertanyaan");
        for (int i = 0; i < (int)daftar_pertanyaan.size(); i++) {
            string pemilik = daftar_pertanyaan[i].nama_pemilik;
            string pertanyaan = daftar_pertanyaan[i].isi_pertanyaan;
            if ((int)pekilik.length() > 13) pekilik = pekilik.substr(0, 10) +
            "...";
            if ((int)pertanyaan.length() > 36) pertanyaan = pertanyaan.substr(0, 33) +
            "...";

            cout << "| " << left << setw(4) << i + 1
                << "| " << setw(15) << pekilik
                << "| " << setw(38) << pertanyaan << "|" << endl;
        }
        tampilGarisBawahTabel(15, 38);

        int nomor_pilihan;
        cout << " Pilih nomor: "; cin >> nomor_pilihan; cin.ignore();
        validasiNomorPilihan(nomor_pilihan, 1, (int)daftar_pertanyaan.size());

        int index_hapus = nomor_pilihan - 1;
        for (int i = 0; i < (int)daftar_pengguna.size(); i++) {
            if (daftar_pengguna[i].profil.nama ==
            daftar_pertanyaan[index_hapus].nama_pemilik)
                kurangiJumlahPertanyaan(&daftar_pengguna[i].jumlah_pertanyaan_dibuat);
        }
        daftar_pertanyaan.erase(daftar_pertanyaan.begin() + index_hapus);
        cout << " Pertanyaan dihapus." << endl << endl;

    } catch (const runtime_error &e) {
        cout << " [ERROR] " << e.what() << endl << endl;
    } catch (const out_of_range &e) {
        cout << " [ERROR] " << e.what() << endl << endl;
    }
}

void editJawabanAdmin() {
    try {
        validasiVectorTidakKosong((int)daftar_pertanyaan.size(), "Data pertanyaan");

        tampilHeaderTabel(28, 28, "Pertanyaan", "Jawaban Sekarang");
        for (int i = 0; i < (int)daftar_pertanyaan.size(); i++) {
            string pertanyaan = daftar_pertanyaan[i].isi_pertanyaan;
            string jawaban = daftar_pertanyaan[i].jawaban_kerang;
            if ((int)pertanyaan.length() > 26) pertanyaan = pertanyaan.substr(0, 23) +
            "...";
            if ((int)jawaban.length() > 26) jawaban = jawaban.substr(0, 23) +
            "...";

            cout << "| " << left << setw(4) << i + 1
                << "| " << setw(28) << pertanyaan
                << "| " << setw(28) << jawaban << "|" << endl;
        }
        tampilGarisBawahTabel(28, 28);
    }
}

```

```

        int nomor_pilihan;
        cout << " Pilih nomor: "; cin >> nomor_pilihan; cin.ignore();
        validasiNomorPilihan(nomor_pilihan, 1, (int)daftar_pertanyaan.size());

        string jawaban_baru;
        cout << " Jawaban baru: "; getline(cin, jawaban_baru);
        validasiInputTidakKosong(jawaban_baru, "Jawaban baru");

        perbaruiJawaban(daftar_pertanyaan[nomor_pilihan - 1].jawaban_kerang,
jawaban_baru);
        cout << " Okree, Jawaban ini berhasil diubah." << endl << endl;

    } catch (const runtime_error &e) {
        cout << " [ERROR] " << e.what() << endl << endl;
    } catch (const out_of_range &e) {
        cout << " [ERROR] " << e.what() << endl << endl;
    } catch (const invalid_argument &e) {
        cout << " [ERROR] " << e.what() << endl << endl;
    }
}

void tambahJawabanKerang() {
    cout << " Jawaban saat ini:" << endl;
    for (int i = 0; i < (int)bank_jawaban.size(); i++)
        cout << " " << i + 1 << ". " << bank_jawaban[i] << endl;
    cout << endl;

    cout << " (Ketik 0 untuk batal hmzz )" << endl;
    string jawaban_baru;
    cout << " Jawaban baru: "; getline(cin, jawaban_baru);

    try {
        if (jawaban_baru == "0") {
            cout << " Dibatalkan." << endl << endl;
            return;
        }
        validasiInputTidakKosong(jawaban_baru, "Jawaban baru");
        bank_jawaban.push_back(jawaban_baru);
        cout << " Jawaban baru ditambahin." << endl << endl;
    } catch (const invalid_argument &e) {
        cout << " [ERROR] " << e.what() << endl << endl;
    }
}

void menuUser(int index_pengguna) {
    int pilihan_menu = 0;
    while (pilihan_menu != 5) {
        cout << "===== " << endl;
        cout << " MENU KERANG AJAIB " << endl;
        cout << " Halo, " << daftar_pengguna[index_pengguna].profil.nama << "!" <<
endl;

        cout << "===== " << endl;
        cout << " 1. Tanya Kerang" << endl;
        cout << " 2. Lihat Histori" << endl;
        cout << " 3. Edit Pertanyaan" << endl;
        cout << " 4. Hapus Pertanyaan" << endl;
        cout << " 5. Keluar" << endl;
        cout << " Pilihan: ";
    }
}

```

```

    cin >> pilihan_menu; cin.clear(); cin.ignore(1000, '\n');
    cout << endl;

    if (pilihan_menu == 1) tanyaKerang(index_pengguna);
    else if (pilihan_menu == 2) lihatHistori(index_pengguna);
    else if (pilihan_menu == 3) editPertanyaan(index_pengguna);
    else if (pilihan_menu == 4) hapusPertanyaanUser(index_pengguna);
    else if (pilihan_menu == 5)
        cout << "   Bye, " << daftar_pengguna[index_pengguna].profil.nama
            << ". PULU PULU PULU! PUJA KERANG AJAIBB!" << endl << endl;
    else cout << "   Pilihan ga valid." << endl << endl;
}
}

void menuAdmin(int index_pengguna) {
    int pilihan_admin = 0;
    while (pilihan_admin != 6) {
        cout << "===== " << endl;
        cout << "           MENU ADMIN KERANG AJAIB           " << endl;
        cout << "   Halo Admin, " << daftar_pengguna[index_pengguna].profil.nama << "!"
<< endl;
        cout << "===== " << endl;
        cout << "   1. Lihat Semua Pertanyaan" << endl;
        cout << "   2. Hapus Pertanyaan" << endl;
        cout << "   3. Edit Jawaban Kerang" << endl;
        cout << "   4. Tambah Jawaban ke Kerang" << endl;
        cout << "   5. Lihat Daftar Pengguna" << endl;
        cout << "   6. Keluar" << endl;
        cout << "   Pilihan: ";
        cin >> pilihan_admin; cin.clear(); cin.ignore(1000, '\n');
        cout << endl;

        if (pilihan_admin == 1) lihatSemuaPertanyaan();
        else if (pilihan_admin == 2) hapusPertanyaanAdmin();
        else if (pilihan_admin == 3) editJawabanAdmin();
        else if (pilihan_admin == 4) tambahJawabanKerang();
        else if (pilihan_admin == 5) lihatDaftarPengguna();
        else if (pilihan_admin == 6)
            cout << "   Bye, Admin " << daftar_pengguna[index_pengguna].profil.nama <<
"!" << endl << endl;
        else cout << "   Pilihan ga valid." << endl << endl;
    }
}
}

```

BAGIAN MAIN

```
int main() {
    srand(time(0));
    inisialisasiBankJawaban();

    cout << "===== " << endl;
    cout << "          SELAMAT DATANG          " << endl;
    cout << "    Tanyakan apapun pada 'KERANG AJAIB'    " << endl;
    cout << "===== " << endl;
    cout << endl;

    int pilihan_awal = 0;
    while (true) {
        cout << "  1. Register" << endl;
        cout << "  2. Login" << endl;
        cout << "  3. Keluar" << endl;
        cout << "  Pilihan: ";
        cin >> pilihan_awal; cin.clear(); cin.ignore(1000, '\n');
        cout << endl;

        if (pilihan_awal == 1) {
            registerPengguna();
        } else if (pilihan_awal == 2) {
            int index_pengguna_aktif = loginPengguna(daftar_pengguna);
            if (index_pengguna_aktif == -1) {
                cout << endl;
                cout << "===== " << endl;
                cout << "  Akses ditolak. 3x percobaan gagal.      " << endl;
                cout << "  Program dihentikan.                      " << endl;
                cout << "===== " << endl;
                return 0;
            }
            cout << endl;
            cout << "  YEAY LOGIN BERHASIL! Bonjour, "
                << daftar_pengguna[index_pengguna_aktif].profil.nama << endl;
            cout << "  Role: " << daftar_pengguna[index_pengguna_aktif].role << endl;
            cout << "===== " << endl << endl;

            if (daftar_pengguna[index_pengguna_aktif].role == "admin")
                menuAdmin(index_pengguna_aktif);
            else
                menuUser(index_pengguna_aktif);

        } else if (pilihan_awal == 3) {
            cout << "  Kerang menunggumu kembali. PULU PULU PULU! PUJA KERANG AJAIBB!"
                << endl << endl;
            break;
        } else {
            cout << "  Pilihan ga valid." << endl << endl;
        }
    }
    return 0;
}
```

External Library

```
#ifndef KERANG_H
#define KERANG_H

#include <iostream>
#include <string>
#include <vector>
#include <stdexcept>
using namespace std;

void validasiInputTidakKosong(const string &teks, const string &nama_field) {
    if (teks.empty()) {
        throw invalid_argument(nama_field + " tidak boleh kosong!");
    }
}

void validasiRole(const string &role) {
    if (role != "user" && role != "admin") {
        throw invalid_argument("Role ga valid! Pilih 'user' atau 'admin'.");
    }
}

void validasiNomorPilihan(int nomor, int batas_bawah, int batas_atas) {
    if (nomor < batas_bawah || nomor > batas_atas) {
        throw out_of_range("Nomor di luar batas! Masukkan antara "
            + to_string(batas_bawah) + "-" + to_string(batas_atas) +
            ".");
    }
}

void validasiNilai(double nilai) {
    if (nilai < 0 || nilai > 100) {
        throw out_of_range("Nilai harus antara 0-100!");
    }
}

void validasiVectorTidakKosong(int ukuran, const string &nama_data) {
    if (ukuran == 0) {
        throw runtime_error(nama_data + " masih kosong!");
    }
}

#endif
```


4. Hasil Output

```
=====
                        SELAMAT DATANG
                Tanyakan apapun pada 'KERANG AJAIB'
=====

1. Register
2. Login
3. Keluar
Pilihan:
```

Gambar 4.1
Tampilan login

```
=====
                        REGISTER
=====
Nama           : Morah
Password (NIM) : 024
Role (user/admin): user
Register berhasil sebagai user. Silakan login.
```

Gambar 4.2 Pilihan Register
Sebagai User

```
=====
                        REGISTER
=====
Nama           : Setri
Password (NIM) : 039
Role (user/admin): admin
Register berhasil sebagai admin. Silakan login.
```

Gambar 4.3 Pilihan Register
Sebagai admin

```
=====
                        LOGIN
=====

Percobaan ke-1 dari 3
Nama      : budi
Password (NIM) : 999
Nama atau NIM salah hmzz.. Sisa: 2x
```

Gambar 4.4 Apabila saat login
nama/nim salah

```
=====
Akses ditolak Well. 3x percobaan gagal.
Program dihentikan.
=====
```

Gambar 4.5 Apabila
sudah 3X salah

```
1. Register
2. Login
3. Keluar
Pilihan: 3

Kerang menunggumu kembali. PULU PULU PULU! PUJA KERANG AJAIBB!
```

Gambar 4.6 Pilihan
Keluar

```
=====
                        SELAMAT DATANG
                        Tanyakan apapun pada 'KERANG AJAIB'
=====

1. Register
2. Login
3. Keluar
Pilihan: 4

Pilihan ga valid
```

Gambar 4.7 Pilihan
diluar dari Pilihan
yang ada di tampilan

```

=====
                        REGISTER
=====
Nama           : Budi
Password (NIM) : 999
Role (user/admin): raja
Role ga valid! pilih dua aja jangan maruk. 'user' atau 'admin'.

```

Gambar 4.8 Apabila Di register memilih role selain admin/user

```

=====
                        LOGIN
=====
Percobaan ke-1 dari 3
Nama           : Norah
Password (NIM) : 024

LOGIN BERHASIL! Halo, Norah
Role: user
=====

```

Gambar 4.9 Berhasil Login sebagai user

```

=====
                        MENU KERANG AJAIB
Halo, Norah!
=====
1. Tanya Kerang
2. Lihat Histori
3. Edit Pertanyaan
4. Hapus Pertanyaan
5. Keluar
Pilihan: █

```

Gambar 4.10
Tampilan Menu
user

```
Tanyakan apapun pada kerang ajaib...
Pertanyaanmu: Apakah bumi bentuknya dinosaurus?

Kerang Ajaib berkata..
>> "Sepertinya iya."
```

Gambar 4.11
Pilihan 1 menu user

Histori milik Norah (2 pertanyaan)

No	Pertanyaan	Jawaban Kerang
1	Apakah bumi bentuknya din...	Sepertinya iya.
2	Apakah Matcha itu rasa ru...	Sepertinya tidak.

Gambar 4.12
Pilihan 2 menu user

```
+-----+
| No | Pertanyaan |
+-----+
| 1 | Apakah bumi bentuknya dinosaurus? |
| 2 | Apakah Matcha itu rasa rumput? |
+-----+
Pilih nomor: 2
Pertanyaan baru: Apakah Bebek itu hewan?
Pertanyaan diperbarui
>> Kerang: "Kerang tidak menjawab pertanyaan itu."
```

Gambar 4.13
Pilihan 3 menu user

```
+-----+
| No | Pertanyaan |
+-----+
| 1 | Apakah bumi bentuknya dinosaurus? |
| 2 | Apakah Bebek itu hewan? |
+-----+
Pilih nomor: 2
Pertanyaan dihapus
```

Gambar 4.14
Pilihan 4 menu user

```
=====
MENU KERANG AJAIB
Halo, Norah!
=====
1. Tanya Kerang
2. Lihat Histori
3. Edit Pertanyaan
4. Hapus Pertanyaan
5. Keluar
Pilihan: 5

Bye, Norah. PULU PULU PULU! PUJA KERANG AJAIBB!
```

Gambar 4.15
Pilihan 5 menu user

```
=====
MENU ADMIN KERANG AJAIB
Halo Admin, Setri!
=====
1. Lihat Semua Pertanyaan
2. Hapus Pertanyaan
3. Edit Jawaban Kerang
4. Tambah Jawaban ke Kerang
5. Keluar
Pilihan: |
```

Gambar 4.16 Tampilan Menu admin

No	Pemilik	Pertanyaan	Jawaban
1	Norah	Apakah bumi bentuknya d...	Sepertinya iya.
2	Miya	Sayur itu Hijau?	Tentu saja!

Gambar 4.17 Pilihan 1
Menu admin

No	Pemilik	Pertanyaan
1	Norah	Apakah bumi bentuknya dinosaurus?
2	Miya	Sayur itu Hijau?

Pilih nomor: 2
Pertanyaan dihapus.

Gambar 4.18 Pilihan 2
Menu admin

No	Pertanyaan	Jawaban Sekarang
1	Apakah bumi bentuknya d...	Sepertinya iya.

Pilih nomor: 1
Jawaban baru: Tidak!
Jawaban berhasil diubah

Gambar 4.18
Pilihan 3 Menu
admin

```

Jawaban saat ini:
1. Mungkin suatu hari nanti...
2. Tidak!
3. Coba lagi nanti.
4. Tentu saja!
5. Kerang tidak menjawab pertanyaan itu.
6. Sepertinya iya.
7. Sepertinya tidak.
8. Sudah jelas jawabannya!

(Ketik 0 untuk batal)
Jawaban baru: Well
Jawaban ditambahkan

```

Gambar 4.18
Pilihan 4 Menu
admin

```

=====
          MENU ADMIN KERANG AJAIB
Halo Admin, Setri!
=====
1. Lihat Semua Pertanyaan
2. Hapus Pertanyaan
3. Edit Jawaban Kerang
4. Tambah Jawaban ke Kerang
5. Keluar
Pilihan: 5

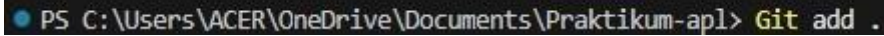
Bye, Admin, Setri!

```

Gambar 4.19
Pilihan 5 Menu
admin

5. Langkah-langkah GIT

5.1 GIT Add

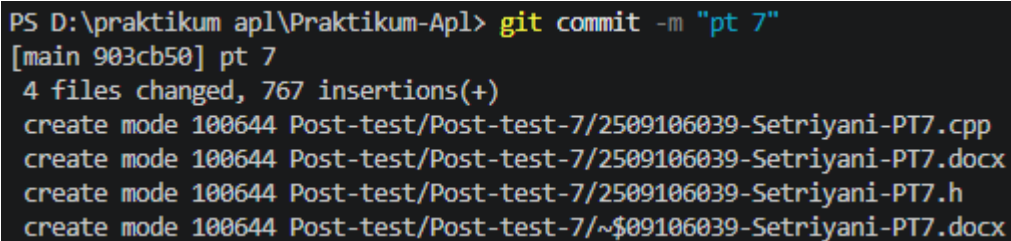


```
PS C:\Users\ACER\OneDrive\Documents\Praktikum-apl> Git add .
```

Gambar 5.1 Git Add

Milih file mana yang mau disimpan di tahap berikutnya.

5.2 GIT Commit

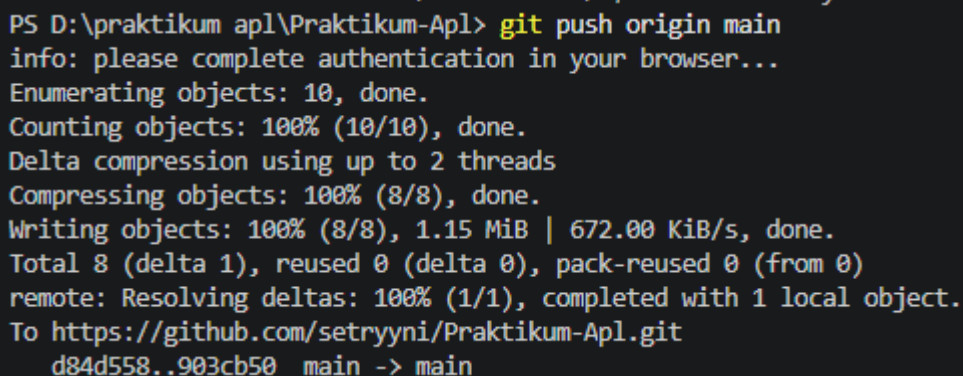


```
PS D:\praktikum apl\Praktikum-Apl> git commit -m "pt 7"
[main 903cb50] pt 7
4 files changed, 767 insertions(+)
create mode 100644 Post-test/Post-test-7/2509106039-Setriyani-PT7.cpp
create mode 100644 Post-test/Post-test-7/2509106039-Setriyani-PT7.docx
create mode 100644 Post-test/Post-test-7/2509106039-Setriyani-PT7.h
create mode 100644 Post-test/Post-test-7/~$09106039-Setriyani-PT7.docx
```

Gambar 5.2 Git commit

Nyimpan perubahan yang udah di add tadi, lengkap sama pesannya.

5.3 GIT Push



```
PS D:\praktikum apl\Praktikum-Apl> git push origin main
info: please complete authentication in your browser...
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 2 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 1.15 MiB | 672.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/setryyni/Praktikum-Apl.git
d84d558..903cb50 main -> main
```

Gambar 5.3 Git Push

Upload semua commit ke repository.